

# THÔNG TIN CHUNG CỦA NHÓM

- Link YouTube video của báo cáo (tối đa 5 phút): TBD
- Link slides (dạng .pdf đặt trên Github của nhóm):

<https://github.com/ttqteo/CS2205.JAN2026/pdf>

- Họ và Tên: Trần Tú Quang
- MSSV: 250201084



- Lớp: CS2205.JAN2026
- Tự đánh giá (điểm tổng kết môn): 9/10
- Số buổi vắng: 0
- Số câu hỏi QT cá nhân: -
- Link Github:

<https://github.com/ttqteo/CS2205.JAN2026>

# ĐỀ CƯƠNG NGHIÊN CỨU

## TÊN ĐỀ TÀI

TỔNG HỢP BẰNG CHỨNG KHAI THÁC DỰA TRÊN PHÂN TÍCH VẾT NHIỄM VI  
PHÂN CHO LỖ HỒNG WEB NPM

## TÊN ĐỀ TÀI TIẾNG ANH

DIFFERENTIAL TAINT-GUIDED PROOF-OF-VULNERABILITY SYNTHESIS FOR  
NPM WEB VULNERABILITIES

## TÓM TẮT

Hệ sinh thái npm với hơn 3 triệu package là nền tảng của phần lớn ứng dụng web hiện đại. Khi một CVE được công bố, việc tự động sinh kịch bản thực thi chứng minh khai thác được (Proof-of-Vulnerability, PoV) giúp nhà phát triển tái lập lỗ hổng và kiểm chứng bản vá. Các công cụ sinh PoV hiện đại cho npm (NodeMedic-FINE, NDSS 2025; PoCGen, 2025; Explode.js, PLDI 2025) đều không dùng bản vá, tức bỏ qua một nguồn thông tin giá trị: bản vá mã nguồn ghi lại chính xác ràng buộc bảo mật mà nhà phát triển đã nhận diện là nguy hiểm. Hệ quả là không gian tìm kiếm payload rộng và tỉ lệ thất bại còn cao trong các trường hợp cần nhiều đầu vào phối hợp hoặc vượt một sanitizer cụ thể.

Đề tài đề xuất phương pháp sinh PoV dựa trên phân tích vết nhiễm vi phân từ bản vá. Với ý tưởng cốt lõi là so sánh đường truyền dữ liệu (đường vết nhiễm) trong mã trước và sau khi vá, trích ra đúng ràng buộc bị thiếu, rồi dùng ràng buộc đó dẫn đường sinh payload khai thác. Vì không gian tìm kiếm được thu hẹp theo ràng buộc từ bản vá, phương pháp nhắm tới các lớp lỗ hổng mà công cụ không dùng bản vá khó xử lý, và cung cấp một nguồn ràng buộc bổ sung cho các nguồn hiện có.

Phạm vi giới hạn ở ba lớp lỗ hổng web phổ biến là Command Injection (CWE-77/78), Path Traversal (CWE-22) và Prototype Pollution (CWE-1321).

Phương pháp được đánh giá trên tập dữ liệu của SecBench.js và so sánh với hai công cụ tiên tiến NodeMedic-FINE (NDSS 2025) và PoCGen (2025) theo độ phủ lớp lỗ hổng, tính bổ sung về nguồn ràng buộc, và hiệu quả sinh PoV. Kết quả kỳ vọng là phương pháp bao phủ được các lớp lỗ hổng mà công cụ không dùng bản vá không xử lý, đồng thời bổ sung các trường hợp mà nguồn ràng buộc khác bỏ sót.

## GIỚI THIỆU

Bảo mật phần mềm trong hệ sinh thái npm đang trở thành vấn đề cấp thiết khi số lượng package tăng nhanh và chuỗi phụ thuộc ngày càng phức tạp. Một lỗ hổng trong package được dùng rộng rãi (như lodash, hàng tỉ lượt tải mỗi tuần) có thể ảnh hưởng đến hàng triệu ứng dụng. Việc tự động sinh PoV giúp xác nhận lỗ hổng là thật và kiểm chứng bản vá có hiệu lực. Đây là nhu cầu thời sự khi số lượng thông báo lỗ hổng npm tăng nhanh hơn năng lực thẩm định thủ công.

**Vấn đề hiện tại.** Các công cụ sinh PoV cho npm đều không dùng bản vá làm nguồn ràng buộc: NodeMedic-FINE lấy ràng buộc từ đồ thị nguồn gốc dữ liệu lúc chạy; PoCGen dựa vào LLM duyệt kết quả CodeQL; Explode.js dùng phân tích tĩnh và thực thi tượng trưng. Thống kê thất bại của NodeMedic-FINE cho thấy 61% hổng do PoV cần nhiều đầu vào phối hợp, 10% do vượt sanitizer, 14% do thiếu ký tự đặc biệt. Đây đúng là những trường hợp mà bản vá thường đã ghi lại ràng buộc bị thiếu qua chốt kiểm tra mới, biểu thức chính quy mới, hoặc kiểm tra kiểu dữ liệu mới.

**Khoảng trống.** Chưa có phương pháp nào dùng khác biệt bản vá làm nguồn ràng buộc cho việc sinh PoV trên npm. Việc trích xuất vết nhiễm vi phân, so sánh đường vết nhiễm trước và sau bản vá để định lượng ràng buộc bị thiếu, cũng chưa được nghiên cứu cho JavaScript/Node.js.

**Đề xuất.** Đề tài xây dựng phương pháp trích xuất chữ ký vết nhiễm vi phân từ bản vá, rồi dùng ràng buộc trích được để dẫn đường sinh PoV. LLM chỉ dùng để cụ thể hóa payload trong không gian ràng buộc đã thu hẹp, không phải để sinh payload tự do.

### Đầu vào / Đầu ra cụ thể.

- Đầu vào: (1) khác biệt bản vá của một CVE npm đã biết, (2) phiên bản package trước khi vá.
- Đầu ra: kịch bản PoV (Node.js) thực thi được, xác nhận lỗ hổng khai thác được trong môi trường cô lập.

## MỤC TIÊU

**Mục tiêu 1, phương pháp trích xuất vết nhiễm vi phân.** Xây dựng phương pháp hình thức trích xuất chữ ký vết nhiễm vi phân từ khác biệt bản vá của CVE npm. Chữ ký gồm bốn thành phần  $\langle \Sigma, \Pi, \Delta, \neg \text{San} \rangle$ :  $\Sigma$  là tập nguồn dữ liệu nhiễm (tham số hàm, req.body...),  $\Pi$  là đường lan của dữ liệu nhiễm từ nguồn tới sink,  $\Delta$  là các đường chỉ tồn tại ở bản lỗi ( $\Delta =$

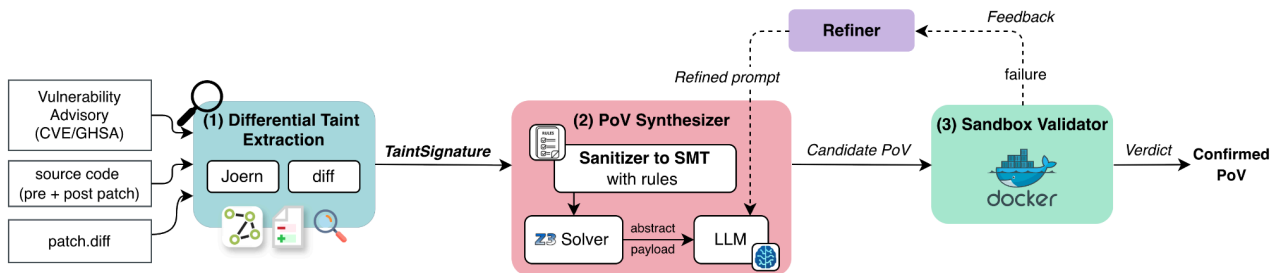
$\Pi_{\text{pre-patch}} \setminus \Pi_{\text{post-patch}}$ ), và  $\neg\text{San}$  là tập kiểm tra bảo mật có ở mã sau vá nhưng thiếu ở mã trước vá.

## Mục tiêu 2, bộ quy tắc dịch sanitizer và quy trình sinh PoV dẫn đường bởi ràng buộc.

Xây dựng bộ quy tắc ánh xạ các mẫu sanitizer phổ biến của npm (kiểm tra khóa, kiểm tra tiền tố, hasOwnProperty, kiểm tra kiểu typeof, biểu thức chính quy, validator tùy biến) sang công thức SMT, kèm phân tích tính đúng đắn và giới hạn (sai về phía dương tính giả hay âm tính giả) cho từng quy tắc. Trên cơ sở đó, quy trình sinh PoV dùng ràng buộc trích được: Z3 giải tìm giá trị thỏa mãn, LLM cụ thể hóa thành payload, và xác nhận trong môi trường cô lập Docker.

**Mục tiêu 3, đánh giá so sánh trên SecBench.js.** Đánh giá quy trình trên tập dữ liệu của SecBench.js, so sánh với NodeMedic-FINE và PoCGen theo độ phủ lớp lỗ hổng và tính bổ sung về nguồn ràng buộc, kèm phân tích các trường hợp thất bại để xác định giới hạn. Thời gian sinh PoV được báo cáo kèm nhưng không đặt làm tiêu chí hơn-thua.

## NỘI DUNG VÀ PHƯƠNG PHÁP



**Bước 1, trích xuất vết nhiễm vi phân từ khác biệt bản vá.** Từ khác biệt bản vá, phương pháp phân tích Code Property Graph (CPG, Yamaguchi et al., S&P 2014) của mã trước và sau vá bằng Joern, tính  $\Delta = \Pi_{\text{pre-patch}} \setminus \Pi_{\text{post-patch}}$  (đường vết nhiễm có trong mã lỗi nhưng bị chặn ở mã đã vá), và trích chốt kiểm tra (guard) mới thành ràng buộc  $\neg\text{San}$ . Phương pháp dùng Joern để xây dựng CPG và trích xuất vết nhiễm.

**Bước 2, sinh PoV có ràng buộc.** Ràng buộc  $\neg\text{San}$  được ánh xạ sang công thức SMT theo bộ quy tắc dịch sanitizer (mục tiêu 2). Z3 giải tìm giá trị thỏa mãn; LLM cụ thể hóa thành payload; PoV được đóng gói dạng kịch bản Node.js. Vì đầu vào đã bị ràng buộc từ bản vá thu hẹp, LLM chỉ tìm trong không gian nhỏ và được điều khiển chặt thay vì sinh payload tự do. Khi PoV chưa kích hoạt, dấu vết thực thi từ môi trường cô lập (đầu ra chuẩn, độ sâu thực thi, thông báo lỗi) được đưa vào lần sinh kế tiếp với temperature giữ ở 0, nên đa dạng hoá

đến từ dấu vết giải thích được thay vì lấy mẫu ngẫu nhiên.

**Bước 3, xác nhận trong môi trường cô lập.** PoV được thực thi trong Docker cô lập với phiên bản package lỗi, gán nhãn khai thác được / không khai thác được bằng bộ phán định (oracle).

**Câu hỏi nghiên cứu.**

- RQ1: Bản vá npm tiết lộ những ràng buộc gì cho việc sinh PoV, và mỗi loại ràng buộc có giới hạn (sai về phía dương tính giả hay âm tính giả) ra sao? Các CVE trong tập đánh giá được phân loại định tính để xây dựng bảng phân loại ràng buộc bản vá, kèm phân tích giới hạn của từng loại.
- RQ2: Quy trình dùng bản vá bổ sung được gì cho các công cụ không dùng bản vá (NodeMedic-FINE, PoCGen) xét về độ phủ lớp lỗ hổng và nguồn ràng buộc?

## KẾT QUẢ MONG ĐỢI

1. **Phương pháp trích xuất:** định nghĩa hình thức chữ ký vết nhiễm vi phân cho npm, kèm thuật toán trích xuất từ khác biệt bản vá và bản hiện thực phương pháp.
2. **Bộ quy tắc và quy trình sinh PoV:** bộ quy tắc các mục dịch sanitizer npm sang SMT kèm phân tích tính đúng đắn và giới hạn từng mục, trong đó ít nhất 3 mục được hiện thực và tích hợp thành quy trình hoàn chỉnh từ khác biệt bản vá đến PoV.
3. **Kết quả so sánh kỳ vọng:** độ phủ lớp lỗ hổng và tính bổ sung được đo và so sánh với NodeMedic-FINE, PoCGen trên cùng tập dữ liệu; kỳ vọng phương pháp phủ các lớp mà công cụ không dùng bản vá không xử lý. Thời gian sinh PoV được báo cáo kèm nhưng không đặt làm tiêu chí hơn-thua vì phụ thuộc thành phần LLM.
4. **Bộ dữ liệu chuẩn:** 30 đến 50 CVE npm có nhãn chuẩn (đường vết nhiễm và PoV xác nhận), phục vụ đánh giá tái lập.
5. **Nghiên cứu trường hợp:** CVE-2019-10744 (lodash, Prototype Pollution) được trình bày trọn quy trình, từ khác biệt bản vá đến PoV thực thi.

Phương pháp khả thi với nguồn lực luận văn thạc sĩ nhờ tái sử dụng công cụ (Joern, Z3, Docker) và dữ liệu (SecBench.js) có sẵn.

## RỦI RO VÀ HẠN CHẾ DỰ KIẾN

- Phụ thuộc LLM: bước cụ thể hóa payload có thể dao động giữa các lần chạy; báo cáo qua nhiều lần chạy để giảm nhiễu. Vì lí do này, đóng góp được đo bằng độ phủ và tính bổ sung thay vì tỉ lệ thành công.

- Độ phủ dữ liệu: kết luận giới hạn trong 3 lớp lỗ hổng và tập SecBench.js.
- Tính đúng đắn bộ quy tắc: mỗi quy tắc cần phân tích riêng; chưa phủ hết mẫu thực tế là hạn chế đã lường trước.

## TÀI LIỆU THAM KHẢO

- [1]. Fabian Yamaguchi, Nico Golde, Daniel Arp, Konrad Rieck: Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE Symposium on Security and Privacy (S&P) 2014: 590-604.
- [2]. Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huang, Wei Zou, Wenchang Shi: MVP: Detecting Vulnerabilities using Patch-Enhanced Vulnerability Signatures. USENIX Security 2020: 1165-1182.
- [3]. Seunghoon Woo, Dongkwan Kim, Seungwon Shin, Heejo Lee: MOVERY: A Precise Approach for Modified Vulnerable Code Clone Discovery from Modified Open-Source Software Components. USENIX Security 2022: 1037-1054.
- [4]. Darion Cassel, Nuno Sabino, Min-Chien Hsu, Ruben Martins, Limin Jia: NodeMedic-FINE: Automatic Detection and Exploit Synthesis for Node.js Vulnerabilities. NDSS 2025.
- [5]. Filipe Marques, Mafalda Ferreira, André Nascimento, Miguel E. Coimbra, Nuno Santos, Limin Jia, José Frago Santos: Automated Exploit Generation for Node.js Packages (Explode.js). Proc. ACM Program. Lang. (PLDI) 2025.
- [6]. Yuxiao Wen, Zhe Liu, Kai Chen, Zhiwu Xu, Shengchao Qin: PoCGen: Automated Generation of Proof-of-Concept Exploits for npm Vulnerabilities. arXiv 2025.
- [7]. Masudul Hasan Masud Bhuiyan, Adithya Srinivas Parthasarathy, Nikos Vasilakis, Michael Pradel, Cristian-Alexandru Staicu: SecBench.js: An Executable Security Benchmark Suite for Server-Side JavaScript. ICSE 2023: 1059-1070.
- [8]. Leonardo Mendonça de Moura, Nikolaj S. Bjørner: Z3: An Efficient SMT Solver. TACAS 2008: 337-340.